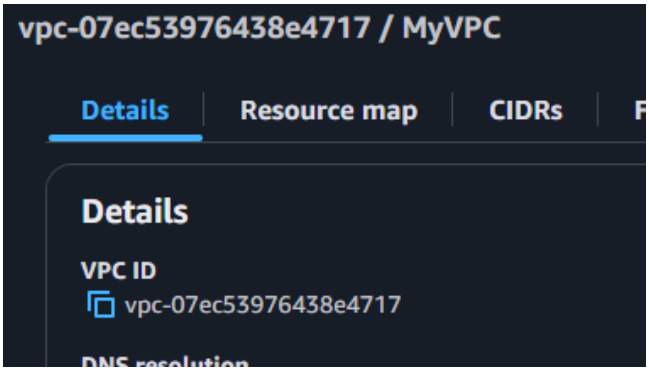
Terraform Challenge -01

- Create VPC

modules > vpc > vpc.tf > resource "aws_route_table_association" "a"

```
resource "aws_vpc" "myvpc" {
  cidr_block = "10.0.0.0/16"

  tags = {
    Name = "MyVPC"
  }
}

resource "aws_subnet" "mysubnet" {
  vpc_id     = aws_vpc.myvpc.id
  cidr_block = "10.0.1.0/24"

  tags = {
    Name = "MySubnet"
  }
}
resource "aws_route_table_association" "a" {
  subnet_id      = aws_subnet.mysubnet.id
  route_table_id = aws_route_table.rt.id
}
```

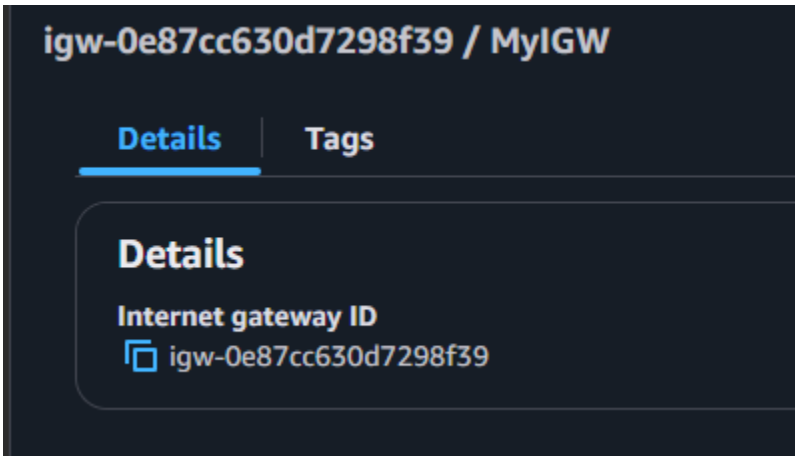


- Create Internet gateway

modules > vpc > igw.tf > resource "aws_internet_gateway" "myigw"

```
resource "aws_internet_gateway" "myigw" {
    vpc_id = aws_vpc.myvpc.id
    tags = {
        Name = "MyIGW"
    }

}
```

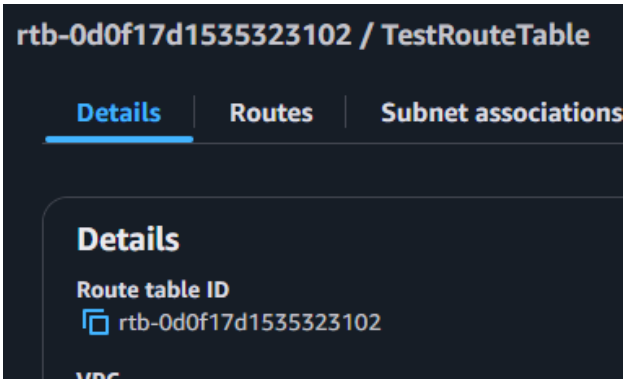
Terraform Challenge -01



- Create Custom Route Table

modules > vpc > rt.tf > resource "aws_route_table" "rt"

```tf
resource "aws_route_table" "rt" {
  vpc_id = aws_vpc.myvpc.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.myigw.id
  }

  tags = {
    Name = "TestRouteTable"
  }
}
```

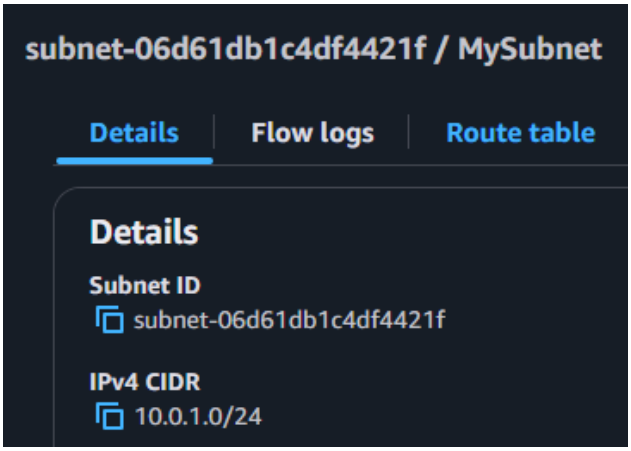


- Create Subnet

Terraform Challenge -01

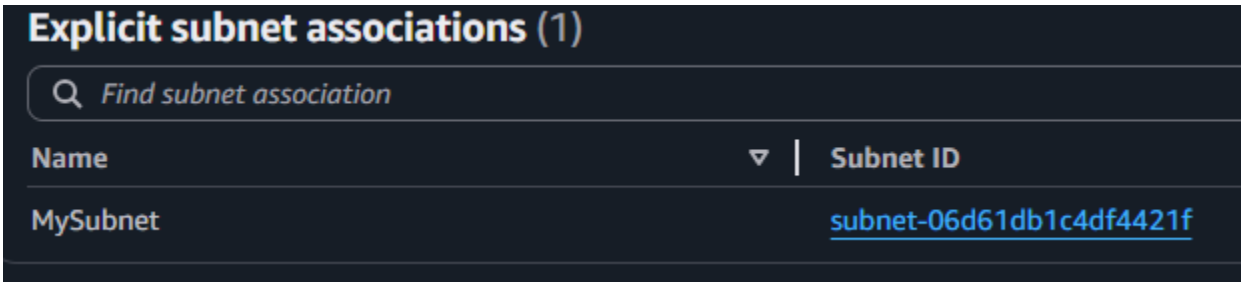
modules > vpc > vpc.tf > resource "aws_route_table_association" "a"

```
resource "aws_vpc" "myvpc" {
  cidr_block = "10.0.0.0/16"

  tags = {
    Name = "MyVPC"
  }
}

resource "aws_subnet" "mysubnet" {
  vpc_id     = aws_vpc.myvpc.id
  cidr_block = "10.0.1.0/24"

  tags = {
    Name = "MySubnet"
  }
}
resource "aws_route_table_association" "a" {
  subnet_id      = aws_subnet.mysubnet.id
  route_table_id = aws_route_table.rt.id
}
```



- Associate subnet with Route Table

```
resource "aws_route_table_association" "a" {
  subnet_id      = aws_subnet.mysubnet.id
  route_table_id = aws_route_table.rt.id
}
```



- Create Security Group to allow port 22.80,443
- protocol = “-1” means all protocols are allowed.

Terraform Challenge -01

modules > ec2 > sg.tf > resource "aws_security_group"

```terraform
resource "aws_security_group" "web_sg" {
  name   = "web-sg"
  vpc_id = var.vpc_id

  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

sg-0c83babc0276f5b1b - web-sg

Details | Inbound rules | Outbou

Details

Security group name

web-sg

Type	Protocol	Port range	Source
HTTPS	TCP	443	0.0.0.0/0
SSH	TCP	22	0.0.0.0/0
HTTP	TCP	80	0.0.0.0/0

Terraform Challenge -01

Type	Protocol	Port range	Destination
All traffic	All	All	0.0.0.0/0

- Create a network interface with an ip in the subnet that was created in step 4

modules > ec2 > ni.tf > resource "aws_network_interface" "web_nic"

```
resource "aws_network_interface" "web_nic" {
  subnet_id       = var.subnet_id
  private_ips     = ["10.0.1.50"]
  security_groups = [aws_security_group.web_sg.id]
  tags = {
    Name = "web-nic"
  }
}
```

Network interface: eni-042e2ee55081d6751 (web-nic)

True

▼ **IP addresses**

Private IPv4 address
10.0.1.50

IPv6 addresses

Public IPv4 address
3.215.30.126

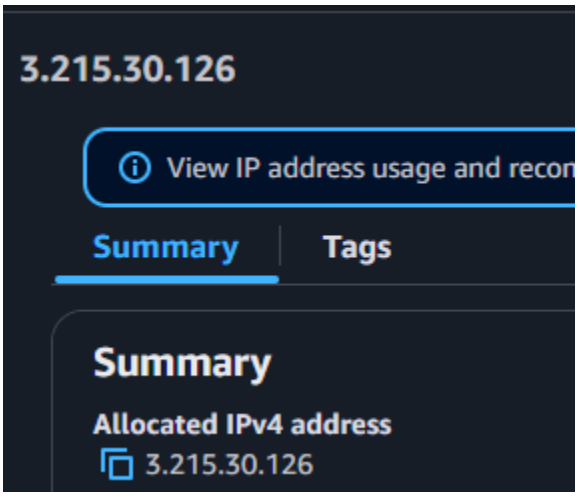
Secondary private IPv4 addresses

- Assign an elastic IP to the network interface created in step 7

modules > ec2 > eip.tf > resource "aws_eip" "eip"

```
resource "aws_eip" "eip" {
  network_interface = aws_network_interface.web_nic.id
  associate_with_private_ip = "10.0.1.50"

  depends_on = [
    aws_instance.web
  ]
}
```

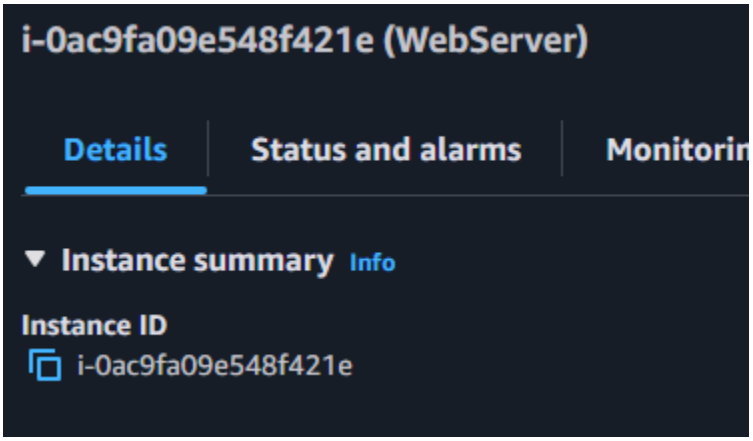
Terraform Challenge -01



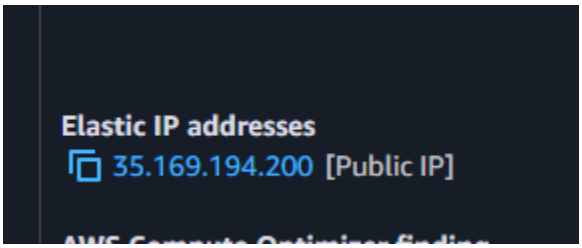
- Create Ubuntu server and install/enable apache2.

modules > ec2 > ec2.tf > resource "aws_instance" "web"

```
resource "aws_instance" "web" {
  ami           = "ami-053b0d53c279acc90"
  instance_type = "t2.micro"
  key_name      = "Linux-key"

  network_interface {
  network_interface_id = aws_network_interface.web_nic.id
  device_index         = 0
}
  user_data = <<-EOF
              #!/bin/bash
              sudo apt update
              sudo apt install apache2 -y
              sudo systemctl start apache2
              sudo systemctl enable apache2
              EOF

  tags = {
    Name = "WebServer"
  }
}
```



Terraform Challenge -01




```
ubuntu@ip-10-0-1-50:~$ systemctl status apache2
● apache2.service - The Apache HTTP Server
     Loaded: loaded (/lib/systemd/system/apache2.service;
     Active: active (running) since Fri 2026-03-06 07:27:
       Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 2260 (apache2)
      Tasks: 55 (limit: 1141)
     Memory: 5.2M
        CPU: 32ms
     CGroup: /system.slice/apache2.service
```

- This was done using modules:

main.tf > module "ec2"

```tf
provider "aws" {
  region = "us-east-1"
}

module "vpc" {
  source = "./modules/vpc"
}

module "s3" {
  source = "./modules/s3"
}

module "ec2" {
  source    = "./modules/ec2"
  vpc_id    = module.vpc.vpc_id
  subnet_id = module.vpc.subnet_id
}
```